A REPORT

ON

SUDOKU SOLVER [GAME]

By

Name of the Student	**Registration No.**
K. Harini	AP24110010123

Prepared in the partial fulfilment of the

Project Based Learning of Course CSE 202 – Object Oriented Programming with C++

SRM UNIVERSITY, AP

NOVEMBER 2025

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to everyone who supported us during the development of our **Sudoku Solver** project. We are thankful to SRM University, AP, for providing the resources and a supportive environment that helped us complete this work.

We are especially grateful to our **Faculty Mentor, Mr. Avinash Trivedi**, for his guidance, helpful feedback, and constant encouragement throughout the project. His support made it easier for us to overcome challenges and improve our work.

We are truly obliged to **Prof. Ch. Satish Kumar** and **Prof. C. V. Tomy**, Dean of the School of Engineering and Sciences (SEAS), for giving us the opportunity, support, and facilities needed to complete this project successfully.

We would also like to thank our **team members** for their cooperation and effort, which helped us handle the technical difficulties faced during the project. Lastly, we would like to thank our **friends, peers, and family members** for their continuous encouragement and support, which played an important role in finishing this work smoothly.

CERTIFICATE

This is to certify that the project entitled "**Sudoku solver [game]**" has been successfully completed by (K. Harini, AP24110010123) as a required academic component of the course CSE 202: Object Oriented Programming with C++ during Semester 3 of Academic Year 2025-26 in the Department of Computer Science and Engineering at SRM University, AP. This work was executed under the guidance of the undersigned and is deemed to have successfully met all course requirements as of 21-Nov-25.

(Signature of the Course Instructor)

Mr. Avinash Trivedi,
Asst Professor,
Department of Computer Science and Engineering
SRM University, AP

ABSTRACT

This project presents a complete and functional Sudoku Solver developed in C++. Sudoku is a popular logic-based puzzle where the goal is to fill a 9×9 grid so that each row, column, and 3×3 sub-grid contains all the digits from 1 to 9 without repetition. Solving these puzzles manually can be time-consuming, so the main aim of this project is to automate the process using an efficient and reliable method.

To achieve this, the solver uses a backtracking algorithm along with recursion. This approach allows the program to explore possible numbers for each empty cell, check whether the placement follows Sudoku rules, and continue building the solution step by step. If it encounters a conflict, the algorithm automatically backtracks, tries a different number, and proceeds until the entire grid is correctly filled.

The final program can solve any valid Sudoku puzzle, from easy to highly challenging ones. It produces accurate results and completes the solving process quickly, demonstrating both efficiency and correctness. Overall, this project highlights how algorithmic techniques can be combined with C++ programming to handle complex logical problems effectively.

TABLE OF CONTENTS

CONTENTS	PAGE. NO.
1.1. COVER PAGE	1
1.2. ACKNOWLEDGEMENTS	2
1.3. CERTIFICATE	3
1.4. ABSTRACT	4
1.5. TABLE OF CONTENTS	5
1.6. INTRODUCTION	6
1.7. METHODOLOGY	7-8
1.8. ALGORITHM	9-10
1.9. CODE	11-13
2.0. FINAL OUTPUT	14-15
2.1. CONCLUSION / RECCOMENDATIONS	16
2.2. REFERENCES	17

INTRODUCTION

Sudoku is a widely popular puzzle that challenges players to use logic and reasoning to complete a number grid. Because of its critical thinking and complexity, solving Sudoku programmatically is an interesting computer science problem involving algorithm design and constraint satisfaction.

One of the reasons Sudoku is so enjoyable is that it combines logic, pattern recognition, and strategic thinking, making it both challenging and satisfying to solve. Playing Sudoku offers several cognitive benefits. It helps improve **critical thinking, concentration, pattern recognition, decision-making, and logical reasoning skills**. As players try different possibilities, eliminate conflicts, and visualize solutions, they naturally develop stronger analytical thinking skills. Sudoku also helps improve memory and encourages patience and persistence, as challenging puzzles often require careful trial and error. For many, it serves as a calming mental activity that reduces stress while enhancing overall problem-solving ability.

This project implements a Sudoku solving tool in C++ that uses recursion and backtracking, a widely known depth-first search (DFS) approach to fill the grid. The system reads a puzzle, validates it, processes it through the solver algorithm, and displays a correct solution.

METHODOLOGY

This project follows a step-by-step method to generate a Sudoku game puzzle, allows the user to play it, and check if the solution is correct. The process includes puzzle generation, difficulty selection, user input checking, and final validation.

1. Generating the Sudoku Grid:

The methodology begins with generating a complete and valid **9×9 Sudoku grid**. To simplify this process, the program first fills the diagonal **3×3 sub-grids** with randomly shuffled numbers from 1 to 9. These boxes do not overlap with other rows or columns, so they can be populated freely without breaking Sudoku rules. Once the diagonal boxes are filled, the remaining cells are solved using a backtracking algorithm. This algorithm attempts to place numbers 1 through 9 in each empty cell and checks whether the placement is valid. If a conflict occurs, the algorithm backtracks and tries another number. This approach continues until a fully valid Sudoku solution is formed.

2. Creating Puzzles for Different Difficulty Levels:

Once a complete solution is generated, the program removes some numbers to create a playable puzzle. The number of removed cells depends on the difficulty chosen:

- **Easy:** 30 cells removed
- **Medium:** 40 cells removed
- **Hard:** 50 cells removed

Removing values introduces blank spaces that the player must fill in. Meanwhile, the program keeps a copy of the original solved grid, which will later be used to validate the user's answers and determine if they have solved the puzzle correctly.

3. Validation of Number Placement:

To maintain the integrity of the Sudoku rules, the program uses an **isSafe()** function that checks whether placing a certain number in a specific cell is valid. The function ensures that the number does not already appear in the same row, the same column, or the 3×3 sub-grid that the cell belongs to. This validation mechanism is essential during three phases: generating the solution using backtracking, creating the puzzle by removing numbers, and checking user inputs while they solve the puzzle.

4. User Interaction and Gameplay:

The program provides an interactive text-based interface through which the player can input their moves. The user enters a row number, column number, and the value they want to place in that position. The program first checks whether the input is within an acceptable range. If the value matches the solution grid's corresponding position, the number is added to the puzzle; otherwise, the program informs the user that the entry is incorrect. After each attempt, the updated puzzle grid is displayed, allowing users to visually track their progress throughout the game.

5. Solution Verification:

The program performs solution verification by comparing the player's current puzzle grid with the original solved grid. It checks each cell one by one to make sure the number placed by the user matches the correct solution.

- If **all values match**, the puzzle is considered fully solved.
- If **any value is incorrect**, the verification fails and the game continues.

Once the entire grid matches perfectly, the program displays a congratulatory message, confirming that the player has completed the Sudoku without any mistakes. This ensures the final answer is both accurate and valid according to Sudoku rules.

6. Use of Randomization for Uniqueness:

The program uses randomization to ensure that every Sudoku puzzle generated is unique. During the grid creation process, the numbers inside the diagonal 3×3 boxes are randomly shuffled before being placed. Because these boxes do not interfere with one another, randomizing them forms the foundation for many possible puzzle variations. This randomness guarantees that the initial structure of the Sudoku grid is never the same across different runs of the program.

In addition to generating the base grid, randomization is also applied when removing numbers to create the final puzzle. Depending on the selected difficulty level, the program randomly chooses several positions in the grid and replaces those values with zeros. As a result, the cells that are removed differ every time, producing a new challenge for the player. This use of randomness ensures a fresh and unpredictable Sudoku experience in each game session.

7. Command-Line Friendly Design:

The program is designed to run smoothly in a simple command-line environment, making it accessible for beginners and convenient for users who prefer lightweight execution without graphical interfaces. Since the entire gameplay is text-based, it works efficiently on almost any system with a terminal. This design choice also ensures that the Sudoku game remains easy to compile, test, and debug using common C++ tools.

- No graphical user interface (GUI) is required to run the game.
- The program runs smoothly on Windows CMD, PowerShell, and Linux terminals.
- It is ideal for VS Code users compiling with **g++** and testing the game through the integrated terminal.
- The lightweight interface makes the program fast, portable, and easy to execute on any machine.
- Simple input/output makes it easy to understand and modify for learning purposes.

ALGORITHM:

Step 1: Start the Program

1. Begin execution.
2. Declare two 9×9 arrays: puzzle and solution.
3. Ask the user to select a difficulty level (easy, medium, hard).

Step 2: Generate a Complete Sudoku Solution

1. Create an empty 9×9 grid initialized with zeros.
2. For each diagonal 3×3 box:
    - Generate a random permutation of numbers 1 to 9.
    - Fill the diagonal box with these shuffled numbers.
3. Use the backtracking algorithm:
    - Find the next empty cell.
    - Try placing numbers 1 to 9.
    - For each number:
        - Check if it is safe using the isSafe() function.
        - If safe, place the number and recursively continue.
        - If not solvable, reset the cell and try the next number.
4. Once the entire grid is filled, copy it to the solution array.

Step 3: Create the Puzzle Based on Difficulty

1. Depending on the selected level:
    - Easy → remove 30 cells.
    - Medium → remove 40 cells.
    - Hard → remove 50 cells.
2. Randomly pick positions and set those cells to zero.
3. Copy this modified grid into the puzzle array.

Step 4: Display the Puzzle

1. Print the Sudoku grid in a formatted layout.
2. Represent empty cells with a dot (".").

Step 5: Accept User Input

1. Prompt the user to enter:
    - Row (1–9)
    - Column (1–9)
    - Value (1–9)
2. If the input is 0 0 0, exit the game.

Step 6: Validate the Move

1. Convert input to 0-based index (row–1 and column–1).
2. Check if:
    - Row, column, or value is out of range
      → Display “Invalid input”.
3. Compare the user’s input with the solution grid.
    - If correct → place the value in the puzzle grid.

- If incorrect → show “Wrong value!”.

Step 7: Check for Completion

1. After each move, compare the puzzle grid with the solution grid.
2. If all cells match:
    - Display the final solved grid.
    - Show “Congratulations! You solved the Sudoku!”
    - End the game.

Step 8: Loop or End

1. If the puzzle is not solved:
    - Print the updated grid.
    - Repeat from Step 5.
2. If the user quits or puzzle is solved:
    - Exit the program.

Step 9: Stop

1. End program execution.

CODE:

sudoku.c++ > solveSudoku(int [N][N])

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <ctime>
using namespace std;

const int N = 9;

// Check if placing num is safe
bool isSafe(int grid[N][N], int row, int col, int num) {
    for (int x = 0; x < N; x++)
        if (grid[row][x] == num || grid[x][col] == num)
            return false;

    int startRow = row - row % 3;
    int startCol = col - col % 3;

    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            if (grid[startRow + i][startCol + j] == num)
                return false;

    return true;
}

```

sudoku.c++ > printGrid(int [N][N])

```cpp
// Backtracking Sudoku solver
bool solveSudoku(int grid[N][N]) {
    int row = -1, col = -1;
    bool emptyFound = false;

    for (int r = 0; r < N; r++) {
        for (int c = 0; c < N; c++) {
            if (grid[r][c] == 0) {
                row = r;
                col = c;
                emptyFound = true;
                break;
            }
        }
        if (emptyFound) break;
    }

    if (!emptyFound)
        return true;

    for (int num = 1; num <= 9; num++) {
        if (isSafe(grid, row, col, num)) {
            grid[row][col] = num;

            if (solveSudoku(grid))
                return true;

            grid[row][col] = 0;
        }
    }
    return false;
}
```

sudoku.c++ > generateSudoku(int [N][N], int [N][N], string)

```cpp

// Print Sudoku grid
void printGrid(int grid[N][N]) {
    cout << "-------------------------\n";
    for (int r = 0; r < N; r++) {
        if (r % 3 == 0 && r != 0)
            cout << "-------------------------\n";
        for (int c = 0; c < N; c++) {
            if (c % 3 == 0) cout << "| ";
            if (grid[r][c] == 0) cout << ". ";
            else cout << grid[r][c] << " ";
        }
        cout << "|\n";
    }
    cout << "-------------------------\n";
}

// Remove cells based on level
void removeCells(int grid[N][N], string level) {
    int count;

    if (level == "easy") count = 30;
    else if (level == "medium") count = 40;
    else count = 50;

    while (count--) {
        int r = rand() % 9;
        int c = rand() % 9;
        grid[r][c] = 0;
    }
}

```

sudoku.c++ > checkWin(int [N][N], int [N][N])

```cpp

// Generate full Sudoku then remove cells
void generateSudoku(int puzzle[N][N], int solution[N][N], string level) {
    int grid[N][N] = {0};

    // Fill diagonal 3x3 boxes with random permutations
    for (int box = 0; box < 9; box += 3) {
        vector<int> v = {1,2,3,4,5,6,7,8,9};
        random_shuffle(v.begin(), v.end());
        int idx = 0;

        for (int r = 0; r < 3; r++)
            for (int c = 0; c < 3; c++)
                grid[box + r][box + c] = v[idx++];
    }

    solveSudoku(grid);

    // Copy solution
    for (int r = 0; r < 9; r++)
        for (int c = 0; c < 9; c++)
            solution[r][c] = grid[r][c];

    removeCells(grid, level);

    // Copy puzzle
    for (int r = 0; r < 9; r++)
        for (int c = 0; c < 9; c++)
            puzzle[r][c] = grid[r][c];
}

```

```
// Check completion
bool checkWin(int puzzle[N][N], int solution[N][N]) {
    for (int r = 0; r < 9; r++)
        for (int c = 0; c < 9; c++)
            if (puzzle[r][c] != solution[r][c])
                return false;
    return true;
}

int main() {
    srand(time(0));

    int puzzle[N][N], solution[N][N];
    string level;

    cout << "Select difficulty (easy / medium / hard): ";
    cin >> level;

    generateSudoku(puzzle, solution, level);

    while (true) {
        printGrid(puzzle);

        cout << "Enter row col value (1-9) or 0 0 0 to quit: ";
        int r, c, v;
        cin >> r >> c >> v;

        if (r == 0 && c == 0 && v == 0) {
            cout << "Game exited." << endl;
            break;
        }

        r--; c--;

        if (r < 0 || r > 8 || c < 0 || c > 8 || v < 1 || v > 9) {
            cout << "Invalid input! Try again." << endl;;
            continue;
        }

        if (solution[r][c] == v) {
            puzzle[r][c] = v;
        } else {
            cout << "Wrong value!" << endl;
        }

        if (checkWin(puzzle, solution)) {
            printGrid(puzzle);
            cout << "Congratulations! You solved the Sudoku!" << endl;
            break;
        }
    }

    return 0;
}
```

FINAL OUTPUT:

```
D:\C++\sudokuProject>g++ sudoku.c++ -o sudoku.exe

D:\C++\sudokuProject>sudoku.exe
Select difficulty (easy / medium / hard): easy
-------------------------
| . 5 3 | 1 . 4 | 7 9 . |
| 8 . 7 | 6 3 9 | 2 . 5 |
| . 9 2 | . 5 7 | 3 6 4 |
-------------------------
| 3 . 6 | 5 4 2 | . 8 . |
| . 8 5 | . . 6 | . . 1 |
| . 7 . | 3 1 . | . . 2 |
-------------------------
| 7 6 1 | 2 . 3 | 5 . 9 |
| . . 4 | 7 9 1 | . . 6 |
| . . 8 | 4 6 5 | . 7 3 |
-------------------------
```

1. **Select the Difficulty Level:**
   The user begins by choosing the difficulty level of the puzzle (Easy, Medium, or Hard). Based on the selected level, a corresponding Sudoku board with dots (.) indicating empty cells is displayed.

2. **Identify Empty Cells:**
   The empty positions in the grid are represented by dots.
   These cells need to be filled using user inputs in the format:
   Row Number – Column Number – Value (r–c–v), where each value ranges from **1 to 9**.

3. **Start Entering Values:**
   The user inputs each value one by one in the format **r c v** (for example: 1 3 5).
   Each entry fills the corresponding empty cell in the grid.

4. **Validate Each Entry:**
   - If the input **matches the correct stored solution**, the value is accepted and placed on the board.
   - If the input **does not match the solution**, the program displays:
     "Try again."
   - If the user enters a row or column number **outside the range 1–9**, or a value not between **1–9**, the program displays:
     "Invalid input."

5. **Continue Filling the Grid:**
   The user keeps entering valid (row–column–value) inputs until all empty cells are filled and the puzzle is complete.

6. **Exit Option:**
   At any time during the input process, the user can exit by entering: **0 0 0**
   This immediately stops further input and terminates the puzzle-solving session.

7. **Complete the Puzzle:**
   Once all entries are correct and the board is completely filled, the program displays the final solved Sudoku grid.

8. **Display Final Output:**
   The fully solved board is shown to the user.

```
Enter row col value (1-9) or 0 0 0 to quit: 6 8 5
-------------------------
| 6 5 3 | 1 2 4 | 7 9 8 |
| 8 4 7 | 6 3 9 | 2 1 5 |
| 1 9 2 | 8 5 7 | 3 6 4 |
-------------------------
| 3 1 6 | 5 4 2 | 9 8 7 |
| 2 8 5 | 9 7 6 | 4 3 1 |
| 4 7 9 | 3 1 8 | 6 5 2 |
-------------------------
| 7 6 1 | 2 8 3 | 5 4 9 |
| 5 3 4 | 7 9 1 | 8 2 6 |
| 9 2 8 | 4 6 5 | 1 7 3 |
-------------------------
Congratulations! You solved the Sudoku!
```

CONCLUSION:

This Sudoku program successfully creates puzzles, solves them, and allows the user to play the game through a command-line interface. It uses backtracking to solve the puzzle and checks every number carefully before placing it, ensuring that the generated Sudoku is always correct. Randomization is used to create different puzzles for each difficulty level: easy, medium, and hard.

Overall, the project demonstrates effective use of core programming concepts such as recursion, arrays, modular functions, randomization, and input validation. Its command-line design makes it lightweight, portable, and easy to use across different environments like Windows CMD, Linux terminal, or VS Code's integrated terminal. The clear structure and readable logic also make the code suitable for educational purposes, helping learners understand how Sudoku solvers and generators function internally.

RECCOMENDATIONS:

1. Introduce a Hint System:

A hint feature could help beginners by revealing one correct value or highlighting possible candidates for a selected cell.

2. Add Timer and Score Features:

Including a timer, move counter, or scoring system can make the game more engaging and competitive for players.

3. Implement Save and Load Game Options:

Allowing users to save their progress and continue later would improve usability, especially for harder difficulty levels.

REFERENCES:

1. Sudoku.com: Sudoku Rules and Strategies
General reference for Sudoku rules and puzzle structure.

2. GeeksforGeeks: "Sudoku Backtracking Algorithm."
Helpful for studying and comparing the logic behind Sudoku solvers.
Retrieved from https://www.geeksforgeeks.org/dsa/sudoku-backtracking-7/

3. Levitin, A. (2012): Introduction to the Design and Analysis of Algorithms. Pearson Education
Used for understanding backtracking and recursive problem-solving techniques.

4. Weiss, M. A. (2013): Data Structures and Algorithm Analysis in C++. Pearson
Reference for C++ implementation of algorithms and array-based structures.

THANK YOU